

Exception Handling Exercise

The following program is intended to divide two numbers and display the result. The code compiles, but something goes wrong when the program is executed.

```
int Divide(int a, int b)
{
    return a / b;
}

Console.WriteLine("Program started.");
int result = Divide(20, 0);
Console.WriteLine("Result: " + result);
Console.WriteLine("Program finished.");
```

Your Task

1. Run or examine the program. What happens when it is executed?
2. Decide whether the problem is a compile-time error or a runtime error.
3. Identify the exception that occurs.
4. Modify the program so that it does not terminate unexpectedly when division by zero occurs.
5. Use try, catch, and finally in your solution.
6. If division by zero occurs, display: "Cannot divide by zero."
7. Make sure "Program finished." is always displayed, whether the calculation succeeds or fails.

Hints

- Think about which line of code may cause an exception.
- Code that may cause an exception should be placed inside a try block.
- You have already seen `DivideByZeroException`. Think about which catch can handle it.
- Which block should contain code that must execute whether an exception occurs or not?

Optional structure hint

```
try { /* code that may cause a problem */ }
catch (/* Which exception? */) { /* handle the problem */ }
finally { /* code that should always execute */ }
```